# SMART TOURISM INTELLIGENCE SYSTEM

Hackathon Blueprint • Features, Risks, Solutions, Tech Stack & MVP Plan

Core idea
Predict where tourist crowds may build up, suggest better times and comparable alternatives, and continuously improve a visitor's route when conditions change.

Suggested pilot: Udaipur, Rajasthan
Team goal: Build a credible working prototype, not a fake 'real-time AI' demo. Clearly show what is measured, what is predicted, and what is estimated.

Prepared for the Out of Scope hackathon team

# 1. Executive Summary

This product is a tourism decision engine, not just a map or chatbot. It helps a visitor answer four questions: Where should I go? When should I go? How crowded might it be? If plans change, what is the best next option?

The strongest differentiator is crowd-aware itinerary re-planning: if a place is predicted to be busy, the system recommends a comparable alternative or shifts the visit time while considering travel time, opening hours, weather, user preferences and the rest of the trip.

Important reality check: A website such as OSIRIS may expose public CCTV feeds in some areas, but camera availability does not mean a verified count of all people at a tourist attraction. A computer-vision count only covers the visible camera view. The prototype should label data as live, recent, historical, predicted, or simulated.

# 2. Problem Statement & Users

Problem: Visitors often arrive at popular attractions at peak times, waste time in queues or traffic, and miss nearby places that could offer a similar experience. Tourism operators also lack a simple view of expected crowd pressure across multiple locations.

Primary users: tourists and families, solo travellers, local guides, destination managers and tourism authorities.

Example: A visitor plans to see City Palace and Lake Pichola. If City Palace is expected to be crowded, the app could recommend a later slot or another nearby attraction, while keeping travel time and opening hours in mind.

# 3. Feature Set

| Feature | What it does / guardrail |
|---|---|
| Crowd level now | Show Low / Medium / High with timestamp and confidence. Only call it live if a real, fresh source exists. |
| Crowd forecast | Estimate crowd level for upcoming time windows (e.g., next 1–3 hours) using historical patterns, day, time, holidays and events. |
| Best time to visit | Recommend lower-pressure time windows that fit opening hours and the visitor's available time. |
| Similar alternatives | Recommend places with a similar type or experience, not random attractions. Include distance and reason. |
| Crowd-aware itinerary | Order attractions to reduce travel and predicted crowd exposure while respecting opening hours and visit duration. |
| Dynamic re-planning | If crowd, weather, closure or travel conditions change, suggest a revised next stop. |
| Map and travel time | Show attractions, route order, estimated journey time and nearby alternatives. |
| Preference-aware recommendations | Use interests such as history, lakes, architecture, shopping, food, family-friendly and budget. |
| Weather and event context | Use forecast, public holidays, festivals and known events as extra signals; show source and update time. |
| Crowd redistribution simulation | A demo mode can estimate what happens if some visitors shift to alternatives. Label this as a simulation, not a real-world guarantee. |
| Data confidence and freshness | Explain whether a result is measured, historical, predicted, or simulated, and show last update. |
| Admin dashboard | Let an authorized manager update opening hours, closures, event notices and verified capacity ranges. |
| Feedback loop | Allow users to report queue/crowd level; treat reports as noisy signals and apply moderation/decay. |

Core MVP (must build): 10–20 attractions, crowd level/forecast, best time, comparable alternatives, route-aware itinerary, and data freshness labels. Stretch: camera-based counting, event integration, admin panel, and crowd redistribution simulation.

# 4. Example User Journey

- User selects Udaipur, available hours, interests and mobility preference.
- System ranks attractions using predicted crowd, travel time, opening hours, weather and fit to interests.
- It proposes a practical itinerary with the reason behind each recommendation.
- Each attraction shows a crowd level plus source/time/confidence (for example: “Predicted: High, 2–4 PM; based on historical pattern”).
- If a selected place becomes unsuitable, the planner suggests a similar place or a later time and recalculates the route.

# 5. Data Needed & How to Obtain It

| Data | Required fields | Possible source |
|---|---|---|
| Attraction metadata | Coordinates, category, opening hours, ticket info, expected visit duration | Official tourism pages, attraction websites, verified manual dataset |
| Travel time / routes | Walking/driving time, distance, route options | Maps/routing API or OpenStreetMap-based routing; verify provider terms |
| Weather | Hourly forecast, rain, heat warnings | Weather API; cache results and show timestamp |
| Calendar signals | Weekday/weekend, public holidays, local events | Holiday calendar, public event feeds, curated local events |
| Historical crowd | Hourly/daily visitor counts or queue length | Tourism department, attraction operator, public reports, research datasets |
| Current crowd | Camera-derived counts, ticket/entry counts, privacy-safe aggregated sensors, user reports | Only with permitted access; public CCTV feeds may be unavailable or unsuitable |

Do not assume: Google Maps popularity, public CCTV, or a website's map layer automatically gives exact real-time occupancy. Check API access, rate limits, licensing, privacy requirements and whether data is available for the target city.

# 6. Main Challenges and How to Tackle Them

| Challenge | Practical mitigation |
|---|---|
| No real-time crowd dataset | Start with historical/weekly patterns and clearly label forecasts. Seek permission from one attraction/operator. Use synthetic data only for UI/demo testing, not as proof of accuracy. |
| No historical counts to train ML | Start with rules or a simple baseline (weekday + hour + holiday/event + weather). Collect opt-in/partner data over time; do not claim an ML model is accurate without testing. |
| Camera blind spots / bad angle | Count only people visible in the frame; use a defined entrance/zone, fixed camera and calibration. Multiple cameras or entry counters are needed for site-wide occupancy. |
| Lighting, occlusion and dense crowds | Use a pre-trained person detector/tracker, test day/night and crowded scenes, report a confidence range, and fall back to “unknown” if feed quality is poor. |
| Predictions can be wrong | Show uncertainty, prediction horizon and last-updated time. Compare forecasts against later observations and measure error; avoid precise headcounts when evidence supports only Low/Medium/High. |
| Alternative place may also be crowded | Score all candidates with the same model and data-quality rules; show 2–3 options and explain why each fits. Avoid sending everyone to one new location. |
| Bad recommendations / long route | Optimize a score that balances crowd, travel time, opening hours, user interests and visit duration. Recompute route after a change. |
| Weather/event data missing | Make these optional signals; show “not available” rather than silently assuming normal conditions. Maintain a manually editable local event list. |
| API cost, limits or outages | Cache data, call APIs from backend, set timeouts and rate limits, monitor quotas, and have a fallback mode with saved attraction metadata. |
| Privacy and CCTV permission | Prefer aggregated counts; use only authorized streams. Do not identify faces, store raw video unnecessarily, or bypass access controls. Check local laws and provider terms. |
| Cold start for a new location | Use category-based similarity, geographic proximity and opening hours first; improve crowd estimates as observations accumulate. |
| Demo credibility | Keep a visible data-source badge: Live / Recent / Historical / Predicted / Simulated. Present a baseline and a test result, not just an animated map. |

# 7. Recommended Tech Stack

| Layer | Recommended choice and reason |
|---|---|
| Frontend | React + Vite; Tailwind CSS; React Router; Leaflet with OpenStreetMap tiles or a permitted map provider. |
| Backend | Node.js + Express.js REST API; input validation; caching; rate limiting. |
| Database | MongoDB for attractions, user preferences, event notices, reports and saved itineraries. Add indexes for location/category/time. |
| Recommendation engine | Start with a transparent weighted scoring function. Example: crowd risk + travel time + opening-hours fit + preference match + weather risk. |
| Forecasting | MVP: rule-based baseline. Later: Python + FastAPI + pandas/scikit-learn for time-series/tabular forecasting if enough labelled data exists. |
| Computer vision (optional) | Python + OpenCV + a person detector/tracker (e.g., YOLO-family model, subject to license and hardware). Process permitted camera feeds; aggregate counts only. |
| Routing | OSRM / OpenRouteService / another licensed routing API; check rate limits and attribution. |
| External data | Weather API; official holiday/event data; authorized attraction/operator feeds. Use adapters so a provider can be swapped. |
| Deployment | Vercel for frontend; Render or another Node/Python host for APIs; MongoDB Atlas. Keep API keys server-side. |
| Testing / monitoring | Vitest or Jest for frontend logic; Supertest for APIs; forecast backtesting scripts; logs, uptime checks and API quota alerts. |

Why this stack fits the team: React/Node/MongoDB lets the team build the product quickly. Python is only needed for a separate forecasting or vision service; do not add it unless that module has a clear purpose.

# 8. Recommendation Logic (Start Simple)

For each candidate attraction, calculate a normalized score. One possible starting formula is:

Score = 0.30 × crowd suitability + 0.25 × interest match + 0.20 × travel-time suitability + 0.15 × opening-hours fit + 0.10 × weather suitability

These weights are illustrative, not validated. Tune them with test cases and user feedback. If a factor is unavailable, either re-normalize the remaining weights or mark the result as lower confidence. For crowd forecasting, begin with a baseline such as the typical crowd level for the same weekday and hour, adjusted by holidays/events/weather only when reliable data exists.

For route planning, generate candidate visit orders and score total travel time, predicted crowd exposure and opening-hour violations. For a small itinerary, trying multiple route permutations is enough; use a route-optimization algorithm only when the number of stops justifies it.

# 9. High-Level Architecture

Client (React map and itinerary) → Express API → data adapters (attractions, routing, weather, calendar, permitted crowd feed) → recommendation/forecast service → MongoDB cache and records → ranked itinerary + explanations. An optional Python service can provide forecasts or camera counts. The frontend should never receive private provider API keys.

Keep the forecast service separate from recommendation logic: forecasting estimates crowd; recommendation chooses what the user should do with that estimate.

# 10. Build Plan for a Hackathon

| Phase | Deliverable |
|---|---|
| Phase 1 — Scope | Pick one area (e.g., Udaipur) and 10–20 attractions. Confirm opening hours, categories and coordinates. |
| Phase 2 — Data | Prepare a small verified dataset. Add 2–3 sample crowd profiles and mark whether each is historical, predicted or simulated. |
| Phase 3 — API | Build attraction, forecast, alternatives and itinerary endpoints. Add data timestamps and source labels. |
| Phase 4 — Product UI | Build map, attraction cards, crowd indicator, best-time view, alternative choices and itinerary. |
| Phase 5 — Logic | Implement baseline crowd rules, similarity scoring and route-aware itinerary scoring. |
| Phase 6 — Demo tests | Test at least 5 scenarios: busy attraction, rain, closure, late arrival, and a preference change. Check API fallback and no-data states. |
| Phase 7 — Explain | Demo one complete journey and show the source/assumption behind each crowd label. Present future work separately. |

# 11. Suggested Backend Endpoints

| Endpoint | Purpose |
|---|---|
| GET /api/attractions?area=udaipur | List verified attraction metadata |
| GET /api/crowd/:attractionId?date=... | Crowd level / forecast + source, timestamp, confidence |
| GET /api/alternatives/:attractionId | Comparable options ranked by fit, distance and predicted crowd |
| POST /api/itineraries/plan | Input time budget, interests, start point and candidate places; return itinerary |
| POST /api/itineraries/replan | Recalculate after crowd/weather/closure or user preference changes |
| POST /api/reports/crowd | Accept user-submitted crowd observation with validation/rate limits |
| GET /api/health | Service health and dependency status |

# 12. How to Prove It Works

- Forecast quality: compare predicted Low/Medium/High with later observations; report accuracy or confusion matrix on held-out dates where labels exist.
- Recommendation quality: manually review whether alternatives match category, distance, opening hours and user intent.
- Itinerary quality: compare total travel time and number of opening-hour violations against a simple nearest-first baseline.
- Reliability: measure API response time, uptime and behavior when weather/routing/crowd data is missing.
- Transparency: every crowd result exposes its data type, source and last update.

# 13. Privacy, Safety & Honest Limitations

- Do not promise exact headcounts unless a validated, permitted counting source supports them.
- Camera-based counting is a separate optional module, not a default capability of a public CCTV map.
- Avoid face recognition and individual tracking; retain only aggregated counts where possible.

- Do not direct all visitors to a fragile or residential area; add capacity/fragility and suitability constraints for alternatives.
- Use official sources where possible and respect API terms, attribution and rate limits.
- When data is unavailable, say so. A good “unknown” state is better than fake real-time data.

# 14. Recommended Scope for the Team

Build this: A Udaipur-focused crowd-aware itinerary planner with 10–20 attractions, explainable Low/Medium/High crowd estimates, best visiting windows, similar alternatives and route re-planning.

Do not make the MVP depend on: scraping arbitrary CCTV, getting exact live headcounts for every attraction, or training a deep model without labels. Treat camera counting as a stretch feature that activates only where a permitted, usable feed exists.

Demo line: “We do not just tell tourists where to go. We help them choose when to go, avoid predicted crowd pressure, and adapt their whole route when conditions change.”

End of blueprint